Calidad de Software aplicada al TFG

Trabajo Práctico Integrador · OptiFull, módulo de interfaz web

Calidad de Software · Mag. Pablo Virgolini

Integrantes: Blason Agostina · Nahmias Arnon · Ortiz Paulina

Nivel declarado: Nivel 3, prototipo avanzado

Entrega: 26 de octubre de 2026

Repositorio: github.com/pauliiortizz/OptiFull · carpeta `Proyecto Calidad/`

1. Contexto, alcance y nivel declarado

OptiFull es el Trabajo Final de Grado del equipo: una plataforma que analiza el video de las cámaras de un local (una estación de servicio con tienda) para obtener métricas de circulación, mapas de calor y alertas de posible hurto, y que identifica los productos en la caja para descontar stock. Está formada por tres procesos que se comunican solo a través de una base de datos compartida en Supabase: el pipeline de video, el pipeline de caja y el **módulo de interfaz web**.

Este TP aplica un proceso de calidad completo a un módulo: el **módulo de interfaz web** (`frontend/` en el repositorio). Lo forman el backend web, una API Flask (módulo 6 del informe de la tesis), y el panel web en React (módulo 7), con el dashboard, los reportes, las alertas, la caja web, la landing y el login. Se eligió porque es la única parte de OptiFull expuesta a internet, la que usa el personal del local todos los días y la que se conecta con los sistemas externos. Los otros dos procesos se siguen probando y midiendo en el mismo pipeline, pero no forman parte del TP.

Nivel declarado: Nivel 3 (prototipo avanzado). El sistema está desplegado en Render, con un entorno de QA y otro de producción; usa Supabase como base de datos en la nube, y el flujo principal funciona de punta a punta.

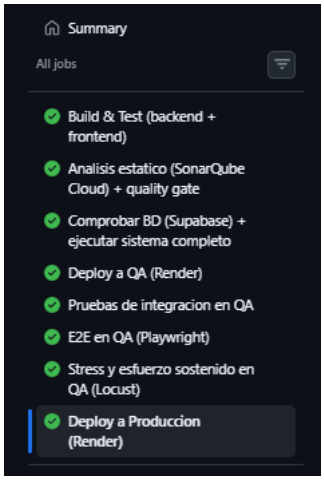
Integración externa. El módulo se conecta con Supabase (Postgres y Storage), con Render (hosting) y con Resend (aviso por email de los pedidos de demo). Cada integración tiene su contrato, sus pruebas con dobles y contra el servicio real, y su paso en el pipeline (sección 4.7 y `integracion-externa.md`).

Todo lo del TP está en la carpeta `Proyecto Calidad/` del repositorio: un documento por eje de la consigna, el plan de pruebas versionado, el registro de defectos y los documentos de apoyo. Este informe resume esos documentos; los números coinciden con el pipeline de GitHub Actions y con SonarQube Cloud.

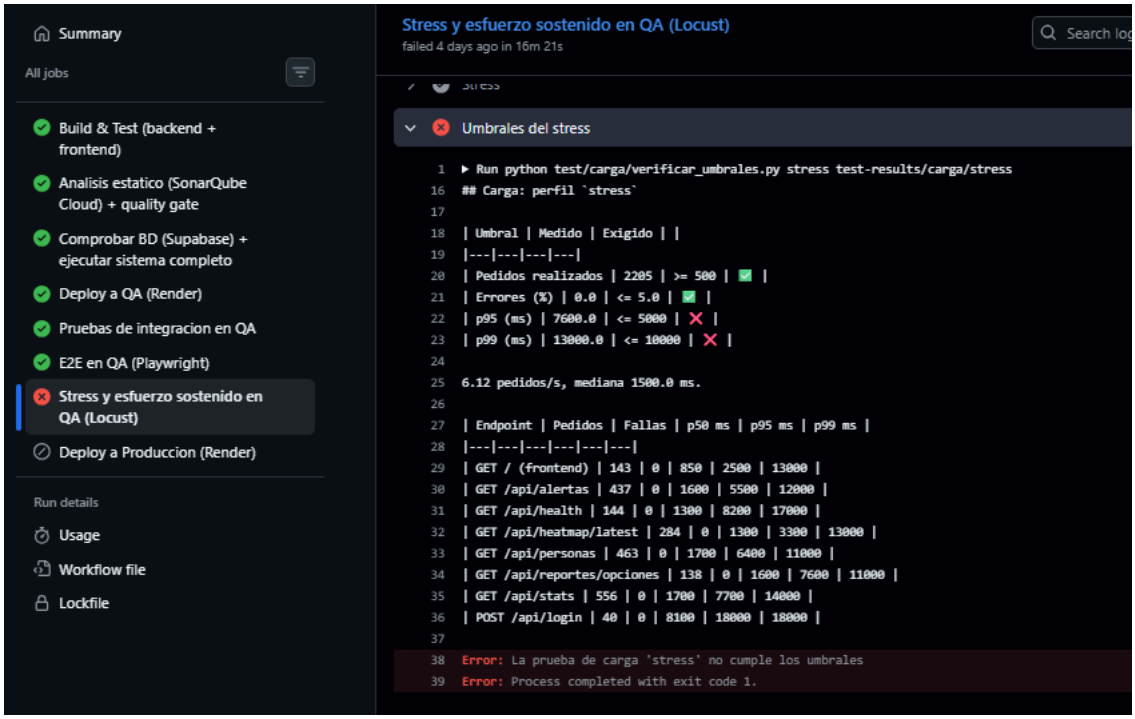
2. El proceso de calidad en el pipeline

El pipeline (`.github/workflows/pipeline.yml`) va de lo más rápido y barato a lo más lento y costoso. Si una etapa falla, se corta ahí y no se despliega lo que sigue.

Etapa	Qué hace	Cuándo
build-test	Compila, Ruff (bloquea errores probables), 817 pruebas unitarias del módulo con cobertura, persistencia contra un Postgres temporal, lint y build del panel	Cada push, también a la rama de trabajo (1,6 min de media)
calidad	Análisis de SonarQube Cloud y quality gate propio	Push a `main`
system-check	Comprueba la base de QA, construye la imagen Docker y corre las pruebas de integración contra el contenedor	Push a `main`
deploy-qa	Despliega en Render el commit exacto y espera a que lo esté corriendo	Push a `main`
integration-qa	Smoke y pruebas de integración contra QA	Push a `main`
e2e-qa	Recorrido de un usuario con Playwright y Chromium	Push a `main`
carga-qa	Stress y esfuerzo sostenido con Locust, con umbrales	Push a `main`
deploy-prod	Despliega en producción el mismo commit y corre el smoke	Push a `main`



- **Ejecución completa en verde:** github.com/pauliiortizz/OptiFull/actions/runs/40210519314
- **Ejecución fallida que detectó un problema real:** .../runs/39643083288. El stress de 40 usuarios dio un p95 de 7,6 s: la API abría una conexión nueva a Supabase en cada pedido.
- **Otra:** .../runs/39495492793. El quality gate frenó código nuevo del panel que había llegado sin tests (73 % de cobertura sobre código nuevo).



3. Evolución de las métricas

La línea base es el commit ef77d4e (20/10/2026), cuando se incorporaron el análisis estático y el quality gate, antes de escribir los tests.

Medida (módulo web)	Línea base	Final (21/10)
Pruebas de la API	casi ninguna	435
Pruebas del panel	7 (un solo archivo)	382
Cobertura de la API (sentencia + decisión)	26 %	97,9 %
Cobertura del módulo en SonarQube Cloud	—	94,6 %
Cobertura sobre código nuevo (proyecto)	—	89,0 %
Duplicación del módulo	—	0,7 % (pero 19,7 % en `video.py`)
Bugs críticos / vulnerabilidades abiertas	— / 10	0 / 0 (calificación de seguridad A)
Defectos encontrados y corregidos	—	20, todos con su test de regresión
Stress de 40 usuarios en QA: p95 / mediana	7 600 ms / 1 500 ms	2 600 ms / 91 ms
Carga sostenida de 15 usuarios en QA: p95	—	140 ms
Funciones con complejidad ciclomática > 10	7	8 (sin cambios de fondo)

Lo que más cambió fueron la cobertura, la cantidad de defectos encontrados y el desempeño bajo carga. La complejidad no cambió: las funciones complejas ya existían, quedaron cubiertas por tests pero no se refactorizaron.

Cobertura Python (sentencia + decision)

Name	Stmts	Miss	Branch	BrPart	Cover
deteccion/config.py	148	16	4	2	88%
deteccion/main.py	486	5	178	11	98%
deteccion/persistencia.py	914	54	316	43	91%
deteccion/pipeline/eventos.py	16	0	8	0	100%
deteccion/pipeline/evidencia.py	136	15	34	3	87%
deteccion/pipeline/fotos.py	88	16	22	4	82%
deteccion/pipeline/heatmap.py	117	1	26	1	99%
deteccion/pipeline/metricas.py	62	0	22	0	100%
deteccion/pipeline/storage.py	29	0	4	0	100%
deteccion/pipeline/tracking.py	241	0	100	11	97%
deteccion/pipeline/video_source.py	251	6	66	5	97%
deteccion/reid/claude_reid.py	82	3	14	1	96%
deteccion/reid/comun.py	169	3	62	3	97%
deteccion/reid/gemini_reid.py	50	3	4	0	94%
deteccion/reid/groq_reid.py	53	2	6	1	95%
frontend/api/reportes_export.py	79	0	18	0	100%
frontend/api/reportes_formatos.py	225	0	72	0	100%
frontend/api/reportes_graficos.py	83	0	18	2	98%
frontend/api/seguridad.py	19	0	0	0	100%
frontend/api/sesion.py	55	0	10	0	100%
frontend/api/stats.py	93	2	22	0	98%
frontend/api/ventas.py	43	0	12	0	100%
frontend/api/video.py	245	4	64	3	98%
TOTAL	**6215**	**178**	**1814**	**112**	**96%**

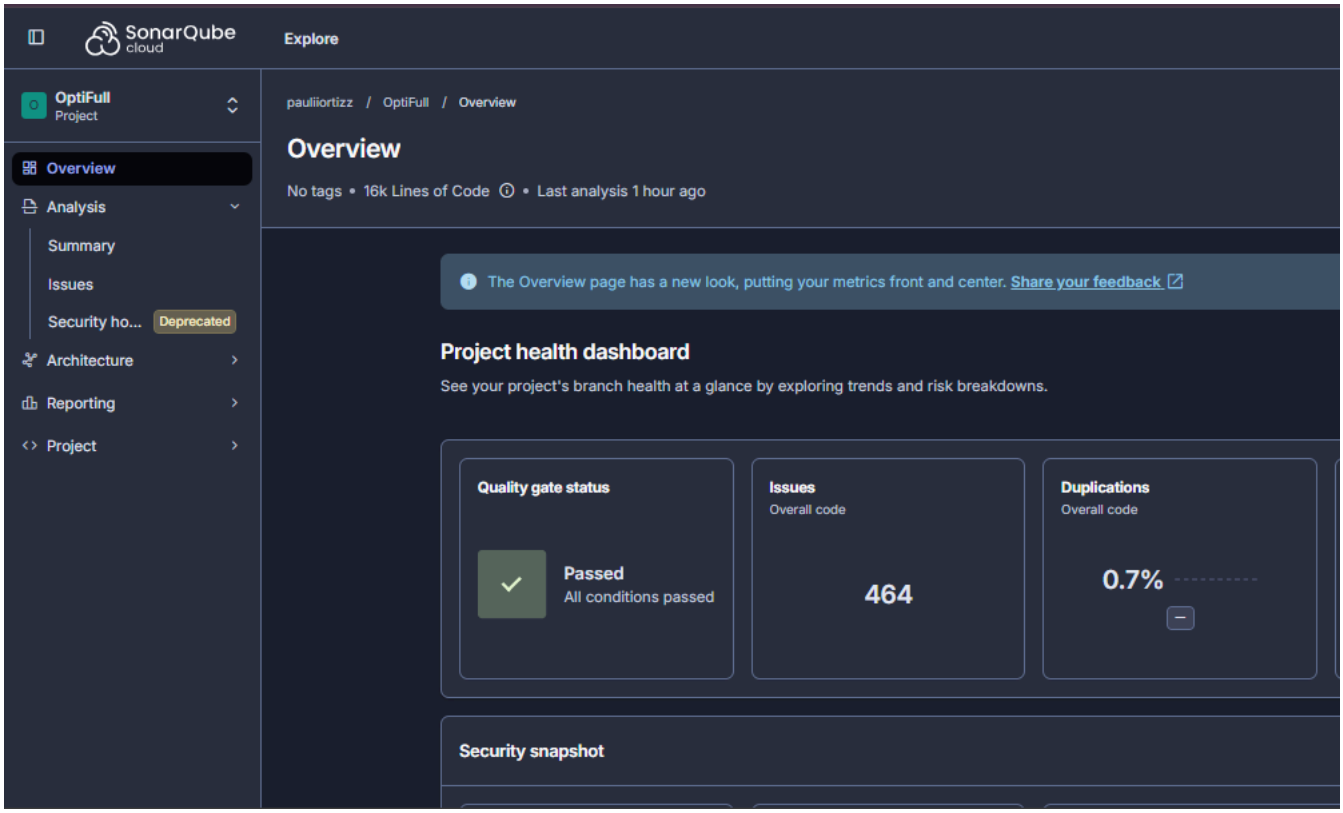
6 empty files skipped.

Vitest Test Report

Summary

- **Test Files:** ✅ **24 passes** · 24 total
- **Test Results:** ✅ **382 passes** · 382 total

Job summary generated at run-time





4. Los ejes de la consigna

4.1 Introducción a la calidad

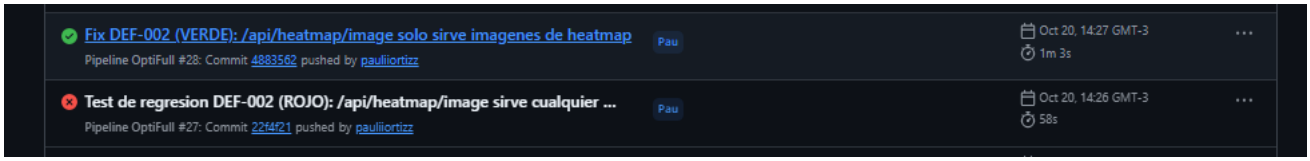
Los objetivos se fijaron desde tres miradas, cada uno con umbral, práctica que lo sostiene y valor medido (`01-introduccion-calidad.md`):

Mirada	Ejemplos de objetivos	Cumplidos
Usuario	p95 ≤ 2 s con la carga esperada (140 ms); nunca datos inventados; 0 errores de JavaScript en el recorrido; todo usable con teclado	3 de 4 (falta accesibilidad: quedan 11 observaciones)
Equipo	Cobertura ≥ 80 % (94,6 %); feedback en ≤ 5 min (1,6 min); 100 % de defectos con regresión (20 de 20); 0 bugs críticos	4 de 4
Organización	0 deploys sin verificar; 0 endpoints con datos sin sesión; 0 USD de costo; volver al verde en menos de un día	3 de 4 (la primera vez tardó 26 h)

El **plan de contingencia** cubre la caída de cada dependencia: cómo se detecta, qué ve el usuario, la respuesta inmediata y la recuperación. Supabase caído: la API responde 503 y el panel muestra "—", y se suspenden los deploys. Render caído: se levanta la misma imagen Docker en la PC del local. GitHub o SonarQube caídos: no se despliega, porque el gate falla cerrado. Armándolo apareció un riesgo nuevo: si el email de los pedidos de demo falla, el pedido queda solo en el disco de Render, que se borra en cada deploy (riesgo R9, abierto).

4.2 Fundamentos del testing

Las pruebas siguen el patrón **Arrange – Act – Assert**. El **criterio de aceptación se fija antes de ejecutar**: los umbrales del stress se escribieron antes de la primera corrida en QA y, cuando no se cumplieron, no se aflojaron: se buscó la causa y se mejoró el sistema. Cada defecto tiene su **contraste rojo/verde**: el test se ve fallar antes del arreglo. Para DEF-002, la API que entregaba el `.env` del servidor, el contraste quedó en el pipeline: .../runs/39466050826 en rojo y .../runs/39466216477 en verde.



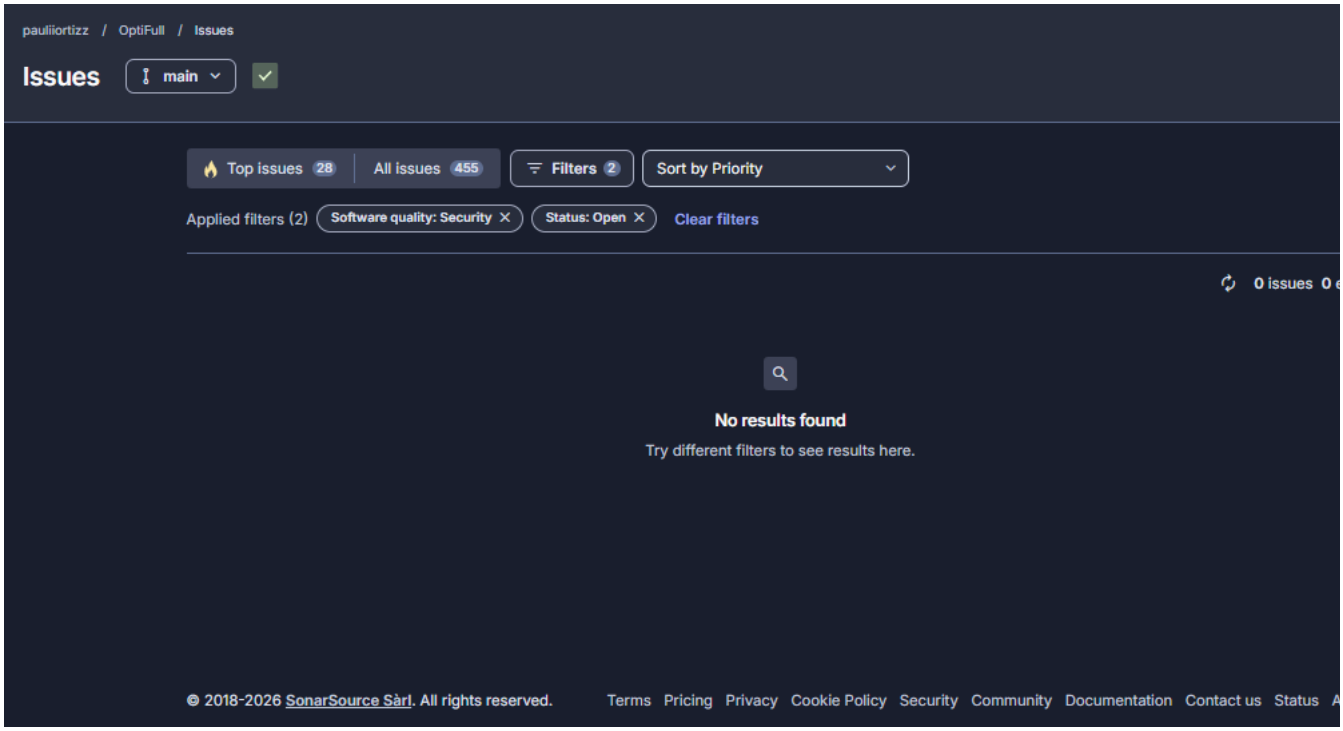
Los **siete principios** se vieron en el TP. Con la API al 98 % de cobertura, el análisis estático igual encontró 5 vulnerabilidades (las pruebas muestran presencia de defectos, no su ausencia). Los defectos se concentraron en cuatro archivos de la API que reciben rutas y URLs del usuario (agrupamiento). Hicieron falta pruebas de otro tipo para encontrar lo que los tests unitarios ya no encontraban: la carga y el E2E (paradoja del pesticida). Y con todo en verde, el stress daba 7,6 s (falacia de la ausencia de errores).

4.3 Modelos de calidad y niveles de prueba

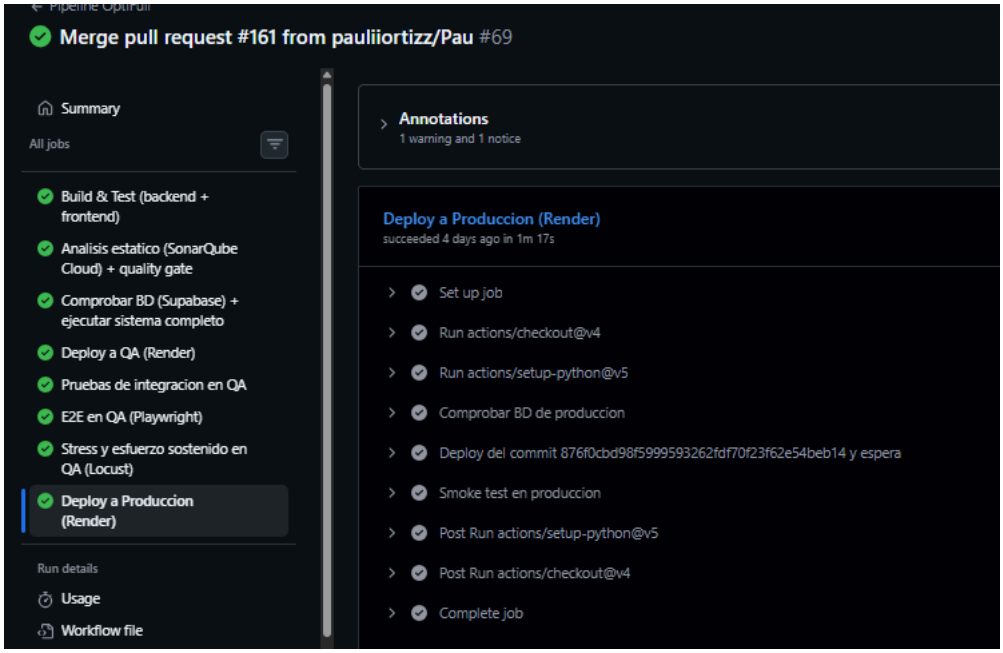
Matriz ISO/IEC 25010 con el formato del caso Knight Capital (`03-modelos-calidad-y-niveles.md`):

Característica	¿Comprometida?	Evidencia	Estado
Adecuación funcional	Sí	PDF que fallaba con algunas versiones de fpdf2; videos y clips con 500; un 0 % inventado	✅
Eficiencia de desempeño	Sí	p95 de 7,6 s bajo stress	✅ con el pool de conexiones
Compatibilidad	Sí	Sin Web Animations no se podía iniciar sesión	✅
Usabilidad	Parcial	Menú sin teclado; mensajes confusos; detalle por hora solo con el mouse	🟡
Fiabilidad	Sí	500 sin base de datos; Dashboard en blanco ante un error	✅
Seguridad	Sí	`.env` descargable; API sin login; SSRF, oráculo de archivos, clickjacking	✅ 0 vulnerabilidades
Mantenibilidad	Parcial	8 funciones con complejidad mayor a 10	🟡
Portabilidad	No	La misma imagen corre en CI, QA y producción	✅

Top 3: Seguridad (los dos defectos críticos, y el módulo está en internet con datos de personas), Fiabilidad (el personal depende del panel y Supabase no lo controlamos) y Eficiencia de desempeño (la que la carga mostró más comprometida y la que llevó a la mejora más grande).



Niveles de prueba: componente (817 pruebas unitarias), integración (23, con Postgres real, el contenedor y QA), sistema (E2E contra QA) y aceptación (E2E por rol, umbrales de carga y smoke en producción). **Despliegue verificado en cada instancia:** contenedor en CI, QA y producción corren el mismo commit, y el pipeline lo comprueba en cada una.



4.4 Métricas y análisis estático

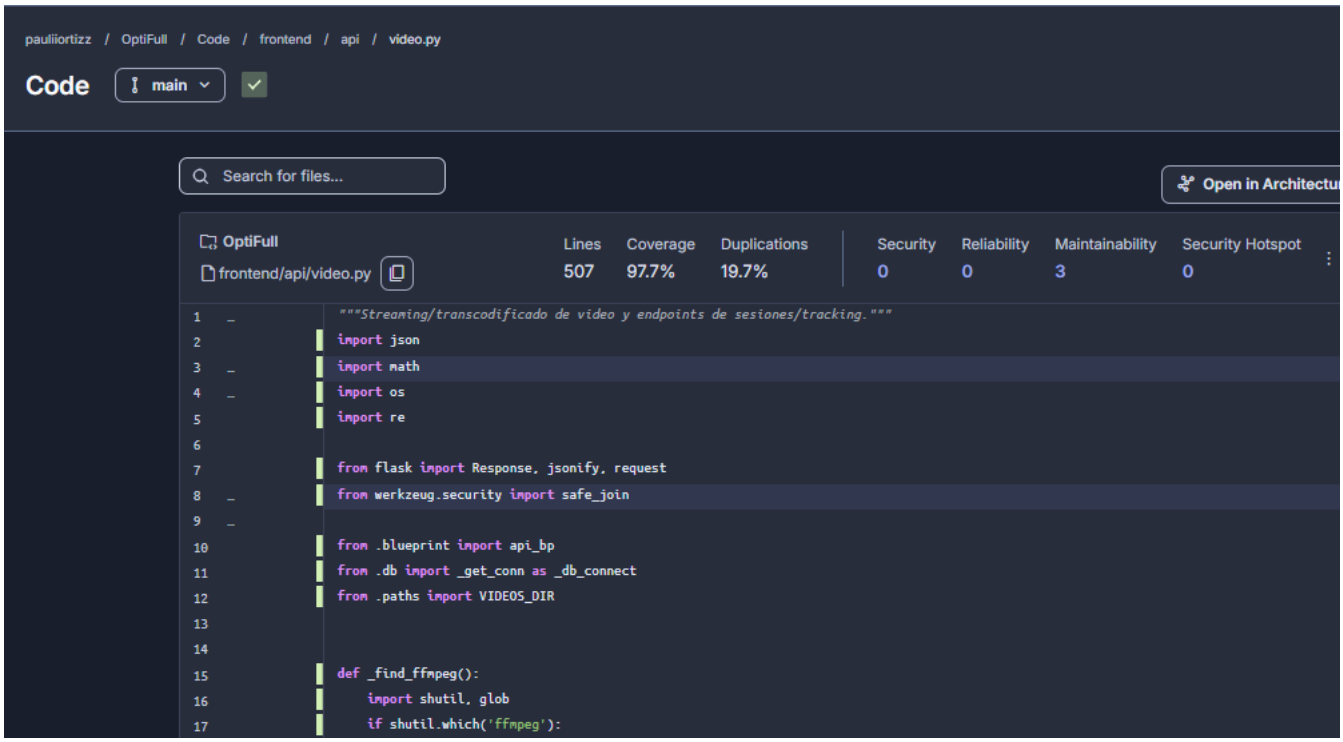
Ocho métricas interpretadas (`04-metricas-y-analisis-estatico.md`), con dónde se concentra cada una. Lo más relevante:

- **Densidad de defectos:** 3,7 defectos por KLOC en la API frente a 1,0 en el panel. La API recibe los datos del usuario, y ahí se concentró la revisión de seguridad.
- **Duplicación:** 0,7 % en el módulo, pero 19,7 % en `video.py`. Un promedio bajo esconde concentraciones.
- **Complejidad:** `generar_pdf()`, con ciclomática 21 y cognitiva 40, está en el rango de alto riesgo.

Métrica de proceso: 2 de 8 ejecuciones de `main` en verde (25 %), y ninguna de las 6 fallas llegó a producción. Tres eran del producto, dos de los propios tests y una fue una hipótesis descartada con datos. El tiempo para volver al verde fue de 26 h la primera vez y de 2 h después. **Métrica de proyecto:** desvío del cronograma de −67 % (se planificaron seis días y se usaron dos).

Hallazgos priorizados:

- **H1:** elementos con clic sin teclado (accesibilidad).
- **H4:** pedidos de demo que se pueden perder.
- **H2:** duplicación en `video.py`.
- **H3:** complejidad de `generar_pdf()`.



Flujo de control, encontrado sin ejecutar el código:

- Una rama defensiva de `_stream_video()` que no se puede cumplir, porque `_rango_pedido()` ya garantiza el rango. La cobertura lo confirma: es una de sus líneas sin cubrir.
- Un `border: 0` que nunca se aplica, porque la propiedad está repetida en el mismo objeto y la segunda lo pisa.

Mejora medida antes y después: el pool de conexiones a Supabase.

Stress, 40 usuarios en QA	Pedidos	Mediana	p95	p99	Resultado
4 hilos, una conexión por pedido	2 205	1 500 ms	7 600 ms	13 000 ms	❌
8 hilos (hipótesis descartada)	2 141	1 600 ms	8 200 ms	14 000 ms	❌
4 hilos, pool de conexiones	3 479	91 ms	2 600 ms	9 300 ms	✅





Locust Test Report

During: 25/10/2026, 13:28:33 - 25/10/2026, 13:34:33 (6 minutes)
Target Host: https://optifull-qa.onrender.com/
Script: locustfile.py

Request Statistics

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	GET / (frontend)	249	0	285.95	15	9893	1375	0.69	0
GET	GET /api/alertas	693	0	478.84	63	11369	932	1.92	0
GET	GET /api/health	203	0	415.84	24	12032	89	0.56	0
GET	GET /api/heatmap/latest	466	0	388.7	29	10381	570	1.29	0
GET	GET /api/personas	712	0	420.52	37	12038	77868	1.97	0
GET	GET /api/reportes/opciones	224	0	277.96	28	8876	2022	0.62	0
GET	GET /api/stats	894	0	498.35	66	12093	427	2.48	0
POST	POST /api/login	40	0	8154.75	1434	15090	123	0.11	0
	Aggregated	3481	0	517.66	15	15090	16533.62	9.65	0

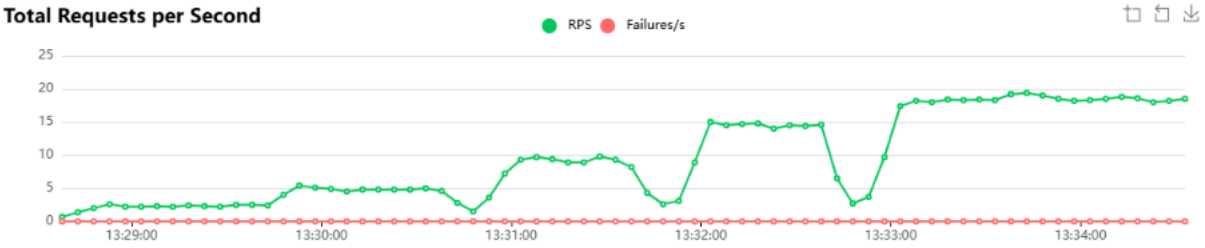
Response Time Statistics

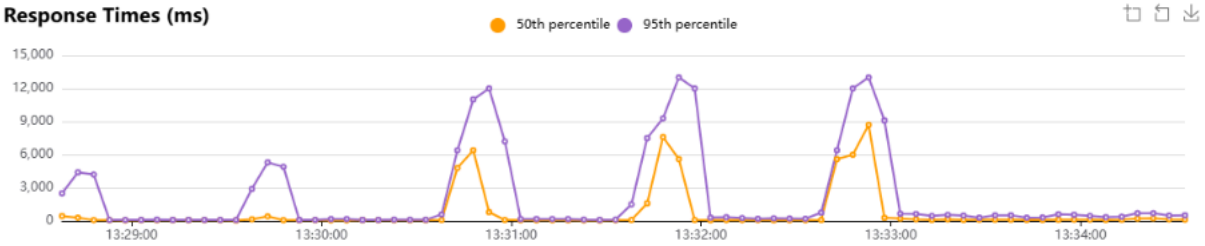
Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	GET / (frontend)	26	30	43	86	210	430	7500	9900
GET	GET /api/alertas	99	120	160	220	450	2000	8700	11000
GET	GET /api/health	39	47	100	160	370	1300	7600	12000
GET	GET /api/heatmap/latest	49	65	120	200	450	1100	8200	10000
GET	GET /api/personas	84	110	160	250	460	1300	8000	12000
GET	GET /api/reportes/opciones	45	56	83	130	270	500	7600	8900
GET	GET /api/stats	140	180	240	350	570	950	10000	12000
POST	POST /api/login	7500	9300	12000	13000	13000	15000	15000	15000
	Aggregated	91	120	160	260	500	2500	9300	15000

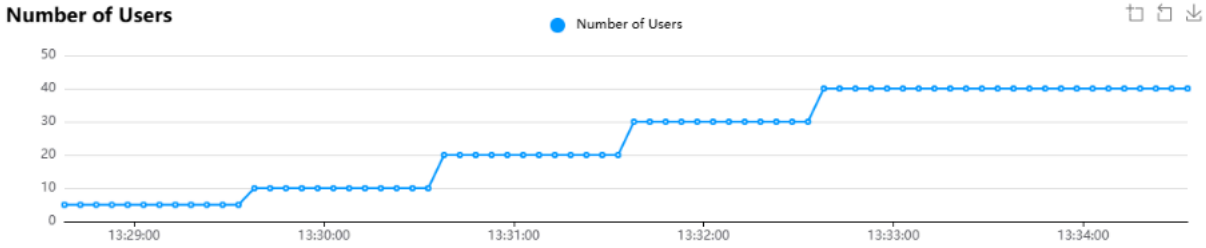
Failures Statistics

# Failures	Method	Name	Message	First Seen	Last Seen

Charts







Final ratio

Ratio Per Class

- 100.0% UsuarioDelPanel
  - 20.0% alertas
  - 6.7% frontend
  - 6.7% health
  - 13.3% mapaDeCalor
  - 6.7% opcionesDeReportes
  - 20.0% personas
  - 26.7% stats

Total Ratio

- 100.0% UsuarioDelPanel
  - 20.0% alertas
  - 6.7% frontend
  - 6.7% health
  - 13.3% mapaDeCalor
  - 6.7% opcionesDeReportes
  - 20.0% personas
  - 26.7% stats

4.5 Pruebas dinámicas

Casos documentados con la plantilla IEEE 829 (`05-pruebas-dinamicas.md`):

- **Clases de equivalencia:** los parámetros de los clips de video (6 clases) y la fuente de la caja web (8 clases, 4 válidas y 4 no válidas).
- **Valores límite:**
- El bloqueo del login: con 4 fallos todavía entra; con 5 se bloquea; a los 301 s se desbloquea. El segundo 300 quedó como pregunta abierta del requisito.
- La cabecera `Range` en el último byte del archivo y en el primero fuera de él.
- **Tabla de decisión:** qué sección del panel ve cada rol (5 reglas).

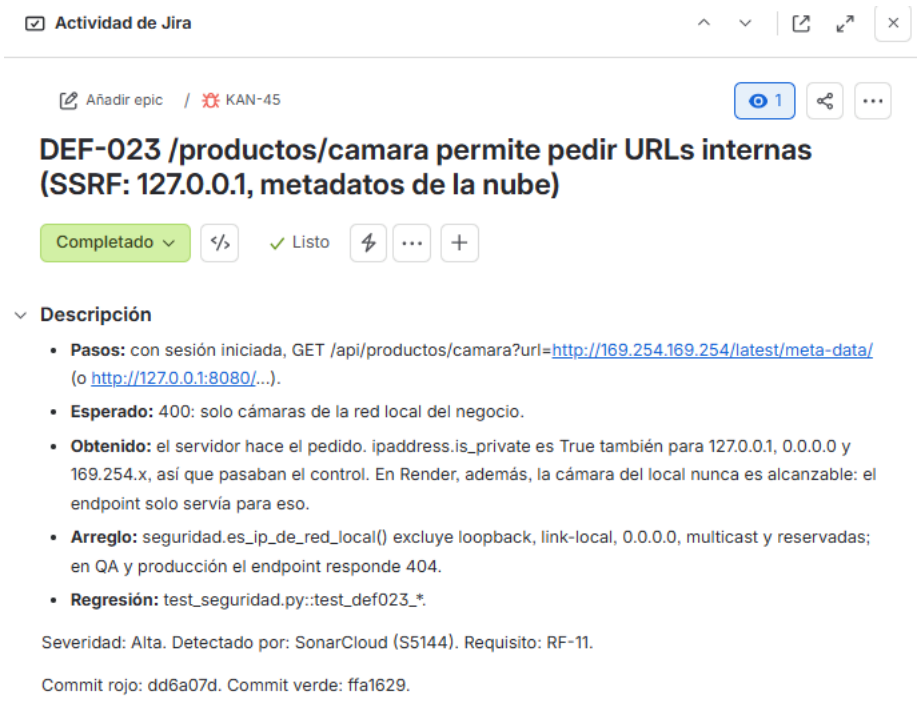
- **Transición de estados:** el formulario de pedido de demo (5 caminos más una transición inválida) y el bloqueo del login.
- **Casos de uso:** los recorridos del administrador y del empleado.
- **Cobertura:** de sentencia 96,8 % y de decisión 90,2 % en el módulo.

4.6 Aseguramiento de la calidad

El **plan de pruebas** está versionado (va por la 3.0) y tiene criterios de entrada, de salida y de suspensión, riesgos y trazabilidad requisito → caso → defecto. **Riesgos:** 9 de producto y 5 de proyecto, priorizados en una matriz de probabilidad × impacto. Los más altos son las dependencias externas (16), la exposición de datos (15) y Supabase caído (15).

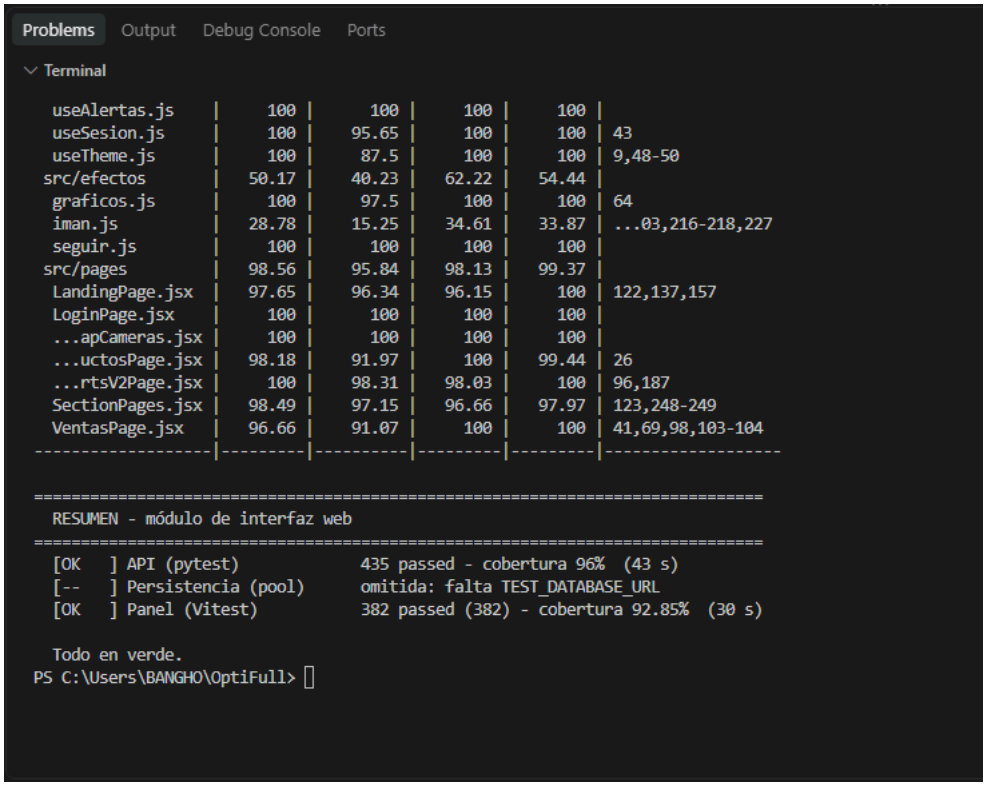
QA y QC. Son QA: los umbrales del gate, el plan, la regla de escribir primero el test que falla, la elección de la herramienta y traer `main` antes de cada PR. Son QC: las pruebas, la carga, el análisis estático, el registro en Jira y la verificación en el pipeline. Hay un ejemplo de lo que el QC no ve sin QA: el E2E pasa en cada deploy, pero con la cuenta de empleado, así que el recorrido del administrador no se prueba en QA. Ningún test va a fallar por eso: es una decisión de alcance, tomada a propósito (el TP se concentra en el rol de usuario común de la interfaz web) y declarada en el informe.

Plantillas de caso y de defecto, con trazabilidad. **Ciclo de vida de un defecto:** "corregido" no es "verificado". Un defecto se cierra en Jira recién cuando el pipeline pasa con su test.



4.7 Automatización del testing

Pirámide: 817 pruebas unitarias y de componentes, 23 de API e integración (13 contra el contenedor y QA, 10 de persistencia), 2 E2E y 2 perfiles de carga. Las **pruebas de API** usan pytest y `requests` (el papel de Postman y Newman en la clase) contra el contenedor y contra QA en Render con Supabase. El **smoke** corre después de cada deploy, a QA y a producción. Hay **un test de regresión por cada uno de los 20 defectos**. **Punto de equilibrio** estimado: unos 11 deploys; entre el 20 y el 21/10 el pipeline corrió 50 veces. No se automatizan la usabilidad ni las pruebas exploratorias.



4.8 Herramientas del ecosistema

Categoría	Herramientas
Gestión de pruebas y defectos	Jira (proyecto KAN) y el plan versionado en el repositorio
Análisis estático	SonarQube Cloud, Ruff, oxlint
Ejecución y registro	pytest, Vitest + Testing Library, Playwright, requests
Cobertura	pytest-cov, V8
Rendimiento	Locust
CI/CD	GitHub Actions, Docker, Render
Diseño / BDD y pruebas específicas	No se usan, con su justificación

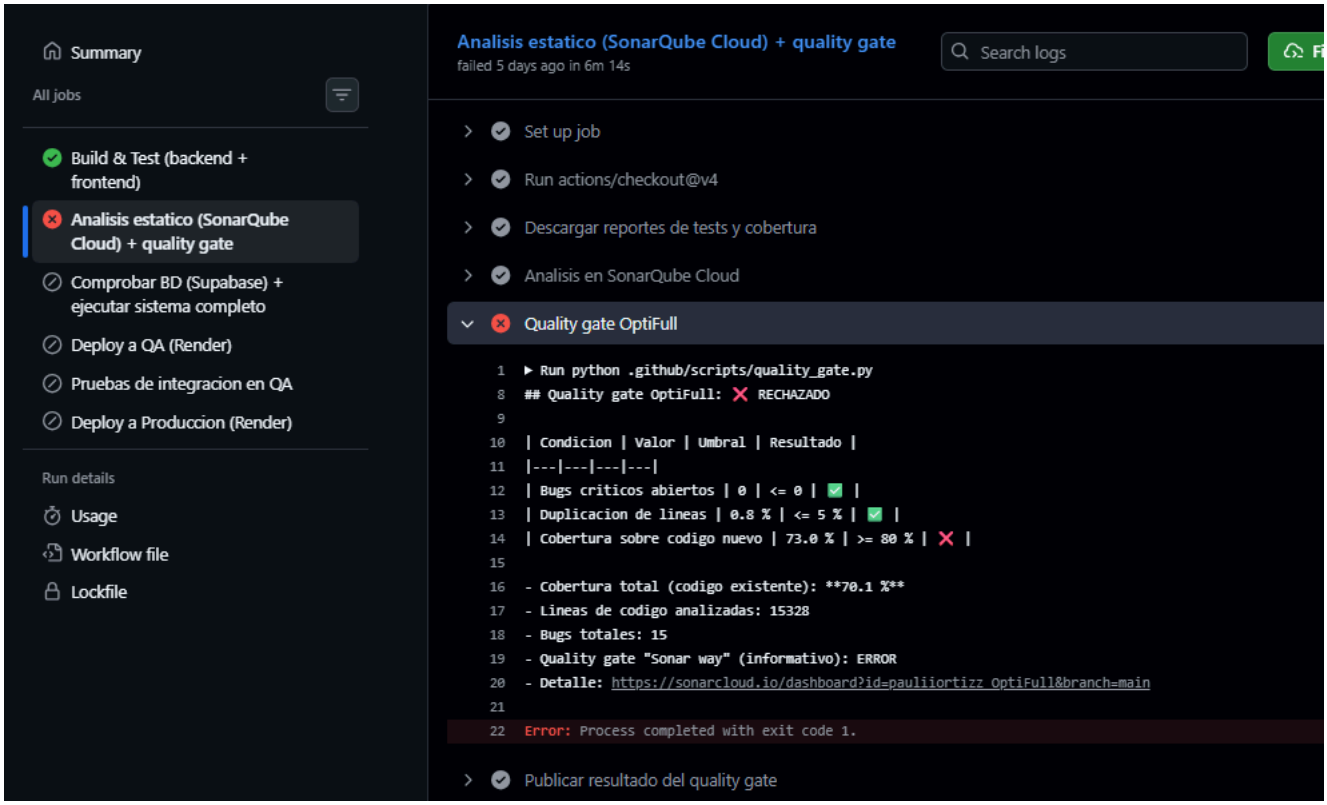
Herramienta nueva: SonarQube Cloud, incorporada con los cinco pasos:

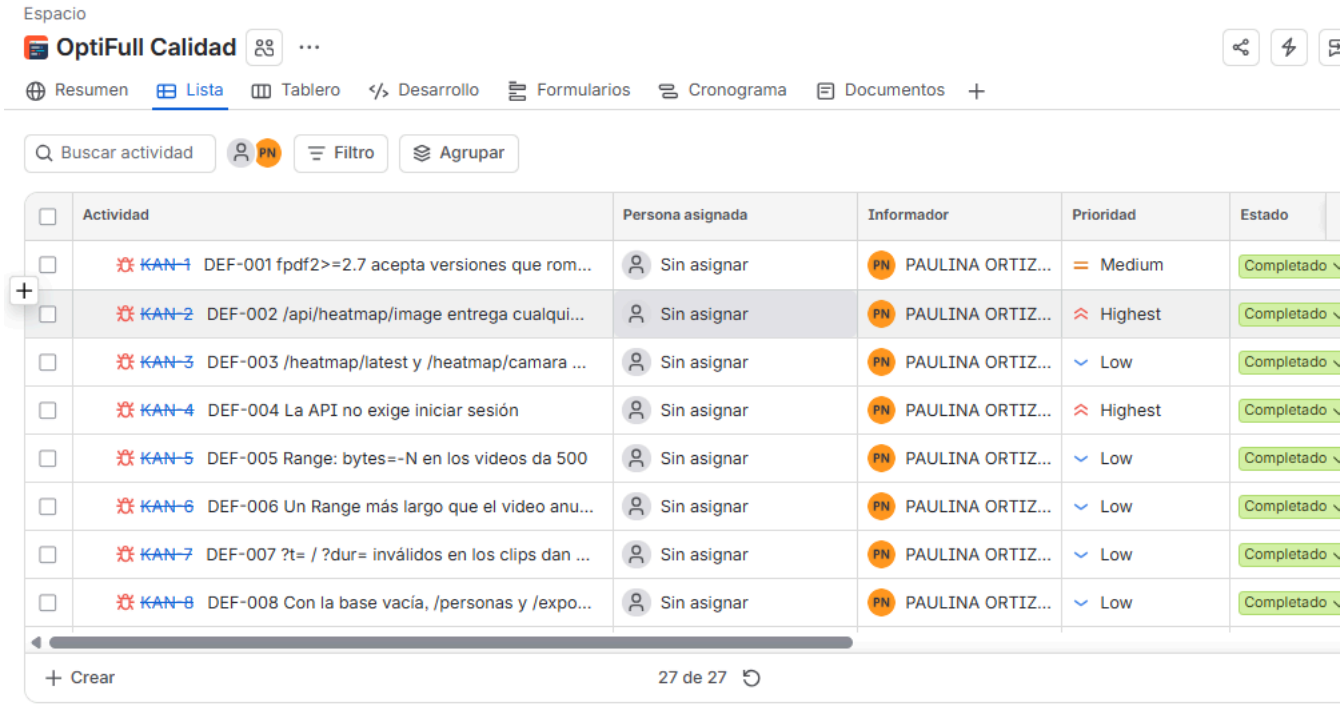
1. Requisitos.
2. Mercado: SonarQube Cloud, SonarQube autoalojado, Codacy y DeepSource.
3. Prueba de concepto: encontró un defecto real.
4. Matriz de decisión ponderada: SonarQube Cloud 4,85 · Codacy 4,00 · DeepSource 4,00 · SonarQube autoalojado 3,80.
5. Despliegue incremental en cinco etapas, hasta el gate bloqueante.

Stress y esfuerzo sostenido en cada deploy, con umbrales (sección 3).

5. Metas del Nivel 3: cumplidas y no cumplidas

Aspecto	Exigido	Estado
Pipeline	Compilación, pruebas, análisis estático, cobertura, gate y reporte automático	✅
Quality gate	Bloquea con bugs críticos, duplicación > 5 % o cobertura nueva < 80 %	✅ Bloqueó dos veces; hoy en OK
Cobertura sobre código nuevo	80 %	✅ 89,0 %
Cobertura sobre código existente	80 %	✅ 94,6 %
Plan de pruebas y riesgos	Versionado, criterios, P × I, trazabilidad	✅
Defectos	Con pasos, esperado y obtenido, vinculados en Jira	✅ 20 del módulo web (27 en Jira con los 7 de otros módulos)
Herramienta nueva	Cinco pasos	✅ SonarQube Cloud
Smoke test	Ejecutado por el pipeline tras el despliegue	✅ En QA y en producción
Integración externa	Flujo E2E automatizado en cada despliegue	✅ E2E con el rol de empleado (usuario común)
Stress y esfuerzo sostenido	Ejecutado, con resultados y umbrales	✅





Las 10 metas se cumplen. Quedaron sin cumplir o parciales:

- La accesibilidad con teclado (11 observaciones).
- Recuperar el verde en menos de un día la primera vez.
- La complejidad, que no bajó.
- El riesgo R9 de los pedidos de demo.
- La revisión de los PR por otro integrante.

6. Análisis crítico

Qué funcionó.

- **Cada falla tuvo una causa.** El pipeline frenó seis veces y ninguna falla llegó a producción. Ninguna se resolvió aflojando un criterio; cada una dejó un cambio en el sistema o en los tests.
- **Medir antes de optimizar.** La primera hipótesis para el stress (más hilos) era razonable y estaba mal, y la prueba de carga lo mostró en una corrida. Sin umbrales fijados de antemano, la diferencia entre 7,6 y 8,2 s se habría leído como ruido.

- **Combinar técnicas.** Los tests unitarios, el análisis estático, el E2E y la carga encontraron defectos de tipos distintos, y ninguna técnica por sí sola los habría encontrado todos.

Qué no funcionó, o no del todo.

- **Los tests también tienen defectos.** Dos de las seis fallas de `main` eran supuestos equivocados de los propios tests: la cuenta de administrador y la carrera del logout. Un test que falla por un error suyo cuesta lo mismo que uno que falla por el producto.
- **Los promedios esconden.** El gate mide la duplicación del proyecto (0,7 %) y no ve el 19,7 % de `video.py`.
- **La calidad interna no mejoró.** La cobertura subió mucho, pero las funciones complejas siguen igual. Están protegidas por tests, pero cada cambio en los reportes sigue siendo caro.
- **El proceso tiene una persona en el medio.** Los PR los mergea quien trabaja en la rama, y las cuentas de los servicios dependen de un integrante (riesgo RP4).

Qué quedó pendiente y por qué.

- **Los pedidos de demo en Supabase (R9) y la accesibilidad (H1):** son cambios del código de la tesis, y el TP no la modifica.
- **El E2E del administrador en QA:** queda fuera del alcance a propósito: el E2E cubre el rol de empleado. El recorrido del administrador se probó localmente y con las pruebas de componentes.
- **Los refactors de `video.py` y `generar_pdf()`:** se priorizaron por debajo, porque ya tienen tests que los protegen.

Sobre el uso de IA. Se usó un asistente como copiloto para los borradores de tests, scripts y documentos. Tres de sus propuestas fallaron en el pipeline y el proceso las detectó: el E2E de administrador, los 8 hilos y la carrera del logout. Es la mejor evidencia de que la IA no reemplaza al proceso de calidad: es una fuente más de cambios que el proceso tiene que verificar (`uso-de-ia.md`).

7. Conclusiones

1. El módulo de interfaz web cumple las **10 metas del Nivel 3**. Cada cambio se prueba automáticamente, desde el commit hasta producción; las integraciones externas (Supabase y Render) se prueban contra el servicio real en cada despliegue, y las decisiones se apoyan en datos.
2. El proceso encontró **20 defectos reales** en el módulo, dos de ellos críticos de seguridad. Todos se corrigieron con su test de regresión, y ninguno se encontró en producción.
3. La mejora más grande no vino de escribir más tests sino de **medir**: la prueba de carga descubrió que la API abría una conexión por pedido, y el pool de conexiones bajó la mediana del stress de 1 500 a 91 ms.
4. Un pipeline en verde no es lo mismo que un sistema de calidad. Cobertura alta y gate en OK convivieron con 19,7 % de duplicación en un archivo, con funciones de complejidad alta y con un riesgo de pérdida de datos. Las métricas y el análisis crítico son los que muestran lo que los tests no buscan.
5. El proceso queda funcionando para el resto de la tesis: cualquier cambio futuro al módulo pasa por las mismas pruebas, los mismos umbrales y el mismo registro de defectos.

Anexo: dónde está cada cosa

Qué	Dónde (en el repositorio)
Índice del TP y cruce con los entregables	`Proyecto Calidad/README.md`
Un documento por eje	`Proyecto Calidad/01-…` a `08-…`
Plan de pruebas versionado	`Proyecto Calidad/plan-de-pruebas.md`
Registro de defectos	`Proyecto Calidad/defectos.md` y Jira, proyecto KAN (optifull.atlassian.net)
Integración externa	`Proyecto Calidad/integracion-externa.md`
Herramienta incorporada y matriz de decisión	`Proyecto Calidad/herramienta-sonarcloud.md`
Pruebas de carga y resultados	`Proyecto Calidad/pruebas-de-carga.md`
Pipeline	`.github/workflows/pipeline.yml`
Código de las pruebas	`test/unitarios`, `test/bd`, `test/integracion`, `test/e2e`, `test/carga`, `frontend/src/**/*.test.*`
Análisis estático	sonarcloud.io, proyecto `pauliiortizz_OptiFull`